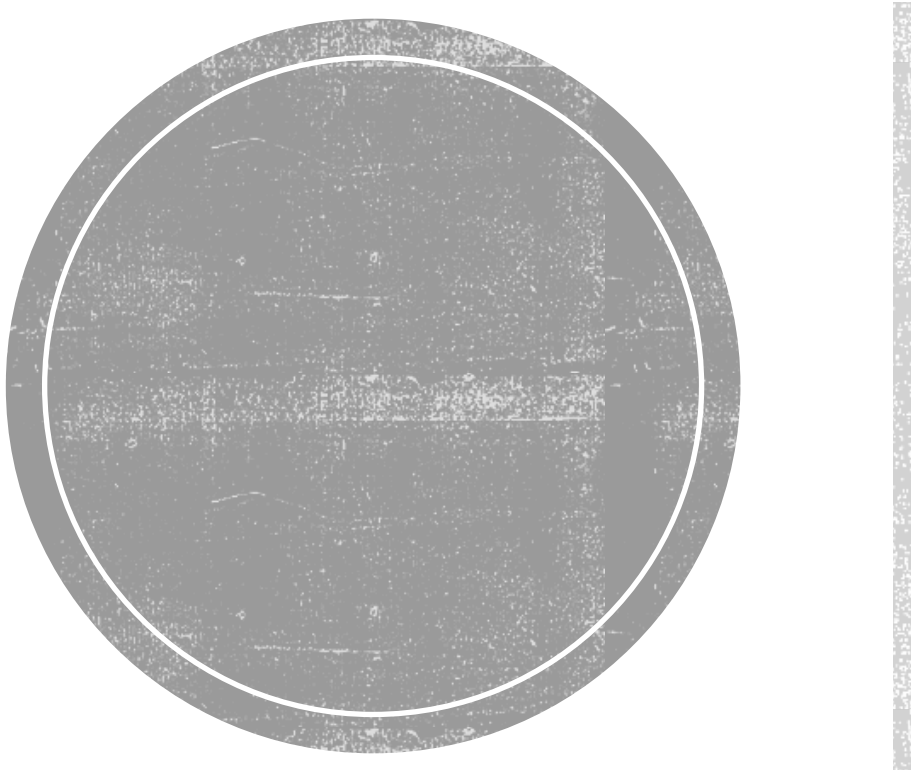
ЛЕКЦІЯ 2. TYPE ANNOTATIONS



ДЛЯ ЧОГО ПОТРІБНЕ АНОТУВАННЯ ТИПІВ

Анотація типів у Python була запроваджена у версії 3.5 як спосіб зробити код зрозумілішим і зменшити кількість потенційних помилок. Хоча Python залишається динамічно типізованою мовою, анотації допомагають розробникам і статичним аналізаторам, таким як MyPy, перевіряти коректність коду.

Зазвичай анотування типів використовується для:

- Зручності сприйняття стороннього коду;
- Зручного редагування коду, коли IDE «підказує» про можливу помилку;
- Відслідковування деяких явних помилок на рівні невідповідності типів.

ДЛЯ ЧОГО ПОТРІБНЕ АНОТУВАННЯ ТИПІВ

```
23  def get_firs_nonzero(items):
24      for item in items:
25          if item:
26              return items
27      return None
```

Нехай потрібно написати функцію, яка приймає масив чисел та повертає перший ненульовий елемент

ДЛЯ ЧОГО ПОТРІБНЕ АНОТУВАННЯ ТИПІВ

```
23  def get_firs_nonzero(items):
24      for item in items:
25          if item:
26              return items
27      return None
```

Неправильна версія

```
23  def get_firs_nonzero(items):
24      for item in items:
25          if item:
26              return item
27      return None
```

Правильна версія

ДЛЯ ЧОГО ПОТРІБНЕ АНОТУВАННЯ ТИПІВ

```
23  def get_firs_nonzero(items):
24      for item in items:
25          if item:
26              return items
27      return None
```

Неправильна версія

```
22  nums = float | int | complex
23
24
25  def get_firs_nonzero(items: list[nums]) -> nums | None:
26      for item in items:
27          if item:
28              return items
29      return None
30
```

Неправильна версія з анотуванням

ОСНОВИ АНОТУВАННЯ ТИПІВ

Анотація типів у Python здійснюється за допомогою двокрапки (:) після імені змінної або параметра функції та вказує очікуваний тип.

```
5   a = 10  # змінна a має тип int
6   b = "Hello"  # змінна b має тип str
7
8   a = a*b  # переприсвоєння не викликає зауважень
9
10  x: int = 10  # змінна x має тип int
11  y: str = "Hello"  # змінна y має тип str
12
13  x = x*y
```

ОСНОВИ АНОТУВАННЯ ТИПІВ

Тут `n: int` означає, що функція factorial2 очікує в якості аргумента ціле число, а синтаксис `-> int` вказує на те, що вона повертає ціле число.

Для перевірки того, чи задано для функції анотування можна скористатись вбудованим параметром `__annotations__`

```
16  def factorial1(n):
17      if n == 1:
18          return 1
19      return factorial1(n - 1) * n
20
21
    3 usages
22  def factorial2(n: int) -> int:
23      if n == 1:
24          return 1
25      return factorial2(n - 1) * n
26
27
28  print(factorial1(5.0), factorial2(5.0), sep='\n')
29  print(factorial1.__annotations__)
30  print(factorial2.__annotations__)
31
```

factorial2() › if n == 1

Run: metodychka

```
C:\Users\oyuso\AppData\Local\Programs\Python\Python311\pyt
120.0
120.0
{}
{'n': <class 'int'>, 'return': <class 'int'>}

Process finished with exit code 0
```

UNION

```python
from typing import Union

number = int | float


def dot1(x: Union[int, float], y: Union[int, float] = 2) -> Union[int, float]:
    return x*y


def dot2(x: int | float, y: Union[int, float] = 2) -> number:
    return x*y


def dot(x: number, y: number = 2) -> number:
    return x*y

```

OPTIONAL

Часто на практиці потрібно задавати опціональні аргументи, для цього можна скористатись або Union, або спеціально вбудованим типом Optional.

```python
from typing import Union, Optional

number = int | float
Str, Str2 = Optional[str], Union[str, None]


5 usages
def dot(x: number, y: number = 2, s: Str = None) -> number:
    if s:
        print(s, x*y)
    return x*y


print(dot(2))
print(dot(2, 4))
print(dot(2, s = 4))
dot(2, 4, 111111)
dot(2, 4, 'result')
```

Run: metodychka

```
C:\Users\oyuso\AppData\Local\Programs\Python\Python311\python.e
4
8
4 4
4
111111 8
result 8
```

ANY

У тих ситуаціях, де аргумент може приймати будь-який із можливих типів, доцільно скористатись анотуванням Any.

```python
from typing import Union, Optional, Any

number = int | float
Str, Str2 = Optional[str], Union[str, None]


5 usages
def dot(x: number, y: number = 2, s: Any = None) -> number:
    if s:
        print(s, x*y)
    return x*y


print(dot(2))
print(dot(2, 4))
print(dot(2, s=4))
dot(2, 4, 111111)
dot(2, 4, 'result')
```

Run: metodychka

```
C:\Users\oyuso\AppData\Local\Programs\Python\Python311\python.exe C:
4
8
4 4
4
111111 8
result 8
```

FINAL

За допомогою анотації Final зручно задавати константи, або ті значення, які не мають змінюватись у ході виконання програми.

```
1   from typing import Final
2
3   MIN_SUM: Final = 1000
4
5   MIN_SUM *= 2
6
```

УЗАГАЛЬНЕНІ ТИПИ (GENERICS)

Python дозволяє створювати узагальнені (generic) типи:

```python
from typing import TypeVar

T = TypeVar('T')


def identity(value: T) -> T:
    return value
```

АНОТУВАННЯ ТИПІВ КОЛЕКЦІЙ

Для версій Python, що передують версії 3.9 анотування таких типів, як list, tuple, dict, set здійснювалось лише за допомогою імпорту з модуля typing.

```python
from typing import List, Tuple, Dict, Set


def process_list(numbers: List[int]) -> Set[int]:
    return set([n * 2 for n in numbers])


pairs: List[Tuple[str, int]] = [("Alice", 25), ("Bob", 30)]
dictionary: Dict[str, int] = {key: value for key, value in pairs}

print(dictionary)

```

LIST, TUPLE, SET, DICT

А починаючи з версії 3.9 можна використовувати саму назву класу, як анотацію.

```
4   def process_list(numbers: list[int]) -> set[int]:
5       return set([n * 2 for n in numbers])
6
7
8   pairs: list[tuple[str, int]] = [("Alice", 25), ("Bob", 30)]
9   dictionary: dict[str, int] = {key: value for key, value in pairs}
10
11  print(dictionary)
```

LIST, TUPLE, SET, DICT

Зауважимо, що об’єкти типу tuple є незмінними, а тому потребують однозначного перелічення типів кожного елемента. У випадку, якщо заздалегідь не відома розмірність кортежу, можна скористатись таким синтаксисом.

```python
from typing import Any, Optional


def read_file(file: str) -> tuple[str, ...]:
    with open(file) as f:
        t = f.readlines()
    return tuple(t)


def read_console() -> tuple[Any, ...]:
    name: str = input('Enter name: ')
    age: int = int(input('Enter age'))
    position: Optional[str] = input('Enter position')
    if position:
        return name, age, position
    return name, age
```

МОДУЛЬ MYPY

MyPy — це статичний аналізатор типів для Python, який дозволяє перевіряти коректність анотацій. Він допомагає знаходити помилки ще до виконання коду.

```
10  def read_console(position: Optional[str] = None) -> tuple[Any, ...]:
11      name: str = input('Enter name: ')
12      age: int = (input('Enter age'))
13
14      if position:
15          return name, age, position
16      return name, age
17
```

```
Terminal:  Local  +
PS C:\Users\oyuso\NaPary\pythonProject\pythonProject\pr_list> mypy metodychka.py
metodychka.py:12: error: Incompatible types in assignment (expression has type "str", variable has type "int")  [assignment]
Found 1 error in 1 file (checked 1 source file)
PS C:\Users\oyuso\NaPary\pythonProject\pythonProject\pr_list>
```

CALLABLE

У Python можна анотувати змінні та параметри, які є об'єктами, які можна викликати, (наприклад, функції або методи), за допомогою Callable з модуля typing

```python
from typing import Callable


1 usage
def execute_function(func: Callable[[int, int], int], a: int, b: int) -> int:
    return func(a, b)


def add(x: int, y: int) -> int:
    return x + y


result = execute_function(add, 2, 3)  # Поверне 5

```

Тут Callable[[int, int], int] означає, що func – це функція, яка приймає два аргументи типу int і повертає значення типу int.

CALLABLE

Анотування за допомогою Callable також можна використовувати з лямбда-функціями.

```
14
15  multiplier: Callable[[int, int], float] = lambda x, y: x / y
16  print(multiplier(12, 4))  # Виведе 3.0
```

АНОТАЦІЇ НА РІВНІ КЛАСІВ

```python
class Person:
    name: str
    age: int
    hobbies: list[str]

    def __init__(self, name: str, age: int, hobbies: list[str]) -> None:
        self.name = name
        self.age = age
        self.hobbies = hobbies
```

АНОТАЦІЇ НА РІВНІ КЛАСІВ

```python
from typing import Optional


3 usages
class Person:
    name: str
    age: int
    hobbies: Optional[list[str]]

    def __init__(self, name: str, age: int, hobbies: list[str] = None) -> None:
        self.name = name
        self.age = age
        self.hobbies = hobbies


Anna: Person = Person('Anna', 23)
Bob: object = Person('Bob', 19, ['reading'])

```

ANY VS OBJECT

Хоч, на перший погляд, типи анотування Any та object виконують одну і ту саму роль, проте між ними є принципова різниця: Any – сумісний з будь-яким іншим типом, а object – несумісний.

```python
from typing import Any

var1: Any = None
s1: str

var2: object = None
s2: str

s1 = var1  # немає проблем при присвоєнні
s2 = var2  # очікує тип str, а отримано object
```

```
Terminal:  Local
PS C:\Users\oyuso\NaPary\pythonProject\pythonProject\pr_list> mypy metodychka.py
metodychka.py:10: error: Incompatible types in assignment (expression has type "object", variable has type "str")  [assignment]
Found 1 error in 1 file (checked 1 source file)
PS C:\Users\oyuso\NaPary\pythonProject\pythonProject\pr_list>
```

АНОТУВАННЯ КЛАСІВ

У даній програмі анотуванням передбачається, що параметр cls_obj функції get_name() буде зсилатись **не на сам клас Person, а на його об’єкти**

```
4  class Person:
5      pass
6
7
8  class Student(Person):
9      pass
10
11
12 def get_name(cls_obj: Person) -> Person:
13     """cls_obj - це посилання безпосередньо на клас"""
14     return cls_obj()
```

```
Terminal:  Local
PS C:\Users\oyuso\NaPary\pythonProject\pythonProject\pr_list> mypy metodychka.py
metodychka.py:14: error: "Person" not callable  [operator]
Found 1 error in 1 file (checked 1 source file)
PS C:\Users\oyuso\NaPary\pythonProject\pythonProject\pr_list>
```

АНОТУВАННЯ КЛАСІВ

Щоб отримати у результаті виконання функції get_name() клас Person, потрібно скористатись Type.

Як бачимо, у ситуації 2 на рівні анотації типів виникають проблеми. Для їх уникнення використаємо згаданий раніше TypeVar.

```
from typing import Type


6 usages
class Person: pass


3 usages
class Student(Person): pass


4 usages
def get_name(cls_obj: Type[Person]) -> Person:
    """cls_obj - це посилання безпосередньо на клас"""
    return cls_obj()


# ситуація 1
person1 = get_name(Person)
student1 = get_name(Student)

# ситуація 2
person2: Person = get_name(Person)
student2: Student = get_name(Student)

```

АНОТУВАННЯ КЛАСІВ

Для цього створюємо деякий загальний тип T, який є об’єктом класу TypeVar. Обов’язково задаємо параметр імені, як правило, його вибирають аналогічним до назви змінної.

Додатково ми задали опціональний параметр bound = Person, що означає, що допустимими є будь-які типи, які зв’язані наслідуванням із класом Person.

```
from typing import Type, TypeVar

5 usages
class Person: pass

3 usages
class Student(Person): pass


T = TypeVar('T', bound=Person)
4 usages
def get_name(cls_obj: Type[T]) -> T:
    """cls_obj - це посилання безпосередньо на клас"""
    return cls_obj()


# ситуація 1
person1 = get_name(Person)
student1 = get_name(Student)

# ситуація 2
person2: Person = get_name(Person)
student2: Student = get_name(Student)

```

РІЗНИЦЯ МІЖ TYPE І TYPEVAR

- Type[T] використовується для позначення класів, які є підтипами T. Це дозволяє працювати з класами як із самостійними об'єктами, наприклад, передавати класи як аргументи функцій.
- TypeVar використовується для створення узагальнених (generic) типів. Це змінна, що представляє довільний тип і дозволяє створювати узагальнені функції або класи.

АНОТУВАННЯ КЛАСІВ: SELF

З версії Python 3.11 можна використовувати Self з модуля typing для позначення того, що метод повертає екземпляр власного класу:

```python
from typing import Self


class Builder:
    def __init__(self, value: str) -> None:
        self.value = value

    def append(self, text: str) -> Self:
        self.value += text
        return self

```

АНОТУВАННЯ КЛАСІВ: СТАТИЧНИЙ ТА КЛАСОВИЙ МЕТОДИ

```python
from typing import Type


2 usages
class Factory:
    @classmethod
    def create(cls: Type["Factory"]) -> "Factory":
        return cls()

    @staticmethod
    def greet() -> str:
        return "Hello!"
```

АНОТУВАННЯ КЛАСІВ: __FUTURE__

Анотовування за допомогою прямого іменування класу в лапках вважається на даному етапі дещо застарілим. Натомість, рекомендовано використовувати модуль __future__.

```
1   from __future__ import annotations
2   from typing import Type
3
4
    2 usages
5   class Factory:
6       @classmethod
7       def create(cls: Type["Factory"]) -> Factory:
8           return cls()
```

АНОТУВАННЯ ВСЕРЕДИНІ КЛАСІВ

```
1 usage
class Point:
    def __init__(self, x: int | float, y: int | float):
        self.x = x
        self.y = y


p = Point(12, 9.6)
p.x = '11'  # середовище не бачить тут помилку
```

```
Terminal:  Local
PS C:\Users\oyuso\NaPary\pythonProject\pythonProject\pr_list> mypy metodychka.py
metodychka.py:8: error: Incompatible types in assignment (expression has type "str", variable has type "int | float")  [assignment]
Found 1 error in 1 file (checked 1 source file)
PS C:\Users\oyuso\NaPary\pythonProject\pythonProject\pr_list>
```

Q&A